CANNBot入门
从0到1生成你的第一个算子



https://gitcode.com/cann/cannbot-skills

CANN

修订记录

版本号	修订日期	修订人	修订页码	修订内容摘要
V2.0	2026/08/26	梁冰龙	全部	初版内容

目录

Part 1 CANNBot概述

Part 2 环境搭建

Part 3 核心工作流

Part 4 实操演示

https://gitcode.com/cann/cannbot-skills

CANN

CANNBot是什么?

核心定位

CANNBot是面向 **CANN开发** 的系列智能体，旨在提升开发效率。本仓库为其提供可复用的 **Skills模块**，目前已实现 **Ascend C / PyPTO/TileLang/Triton/CATLASS 算子开发全流程覆盖**，未来将拓展至CANN更多技术领域。

核心价值

- 标准化开发工作流
- 自动设计方案、编写代码、审查质量
- 输出包含Kernel和Tiling的可编译代码

目标用户

- CANN社区开发者
- Ascend C /PyPTO/TileLang/Triton/CATLASS 算子开发者
- 昇腾NPU平台AI应用开发者
- 希望贡献Skills/Agents的社区贡献者

80+

个Skills

覆盖算子开发全流程

7

个阶段

开发周期从天级缩短到小时级

3

层架构

Teams→Agents→Skills

https://gitcode.com/cann/cannbot-skills

CANN

CANNBot三层架构

T TEAMS（应用编排层）

应用编排

ops-direct-invoke

Ascend C Kernel直调开发流程

pypto-op-orchestrator

PyPTO算子开发流程

ops-registry-invoke

Ascend C 自定义算子开发流程 规划中）

A AGENTS（角色执行层）

角色执行

architect/analyst

方案设计/需求分析

developer

代码开发

reviewer

代码检视

perf-tuner/tester

性能调优/代码测试 规划中）

S SKILLS（知识能力层）

知识能力

npu-arch

NPU架构

tiling-design

Tiling设计

api-best

API最佳实践

precision

精度调试

runtime

运行时调试

编排层：Teams编排Agents，定义开发流程和协作模式

执行层：Agents绑定Skills，执行具体的开发任务

能力层：Skills提供知识能力，支撑Agent执行任务

CANNBot Skills全景图

知识库类

npu-arch
NPU架构知识与芯片映射

ascendc-tiling-design
Tiling设计方法论

ascendc-api-best-practices
API使用最佳实践

ascendc-docs-search
API文档索引+在线搜索

ops-precision-standard
算子精度标准，atol/rtol标准

工具辅助类

ascendc-env-check
NPU设备查询与环境验证

ascendc-task-focus
长任务聚焦防迷失

调试测试类

ascendc-precision-debug
精度调试与症状速查

ascendc-runtime-debug
运行时错误码解析

ops-profiling
算子性能采集分析

测试开发类

ascendc-st-design
ST测试用例设计

ascendc-ut-develop
UT开发与覆盖率增强

ascendc-code-review
代码检视规则

ascendc-whitebox-design
白盒测试用例生成

工程模板类

ascendc-registry-invoke-to-direct-invoke
注册算子直调改造模板

ascendc-direct-invoke-template
Kernel直调工程模板

80+

Skills

覆盖算子开发全流程

知识库 | 调试测试 | 测试开发 | 工程模板 | 工具辅助

目录

Part 1 CANNBot概述

Part 2 环境搭建

Part 3 核心工作流

Part 4 实操演示

https://gitcode.com/cann/cannbot-skills

CANN

环境要求与前置条件

硬件要求

昇腾NPU设备	Ascend 910及后续系列产品
内存	建议16GB以上
存储	建议50GB可用空间

软件要求

CANN版本	8.0.RC2及以上
操作系统	EulerOS 2.10 / CentOS 7.6 / Ubuntu 18.04+
CLI工具	OpenCode、Claude Code等

环境检查清单

- **NPU设备就绪**
  npu-smi info 能正常输出
- **CANN已安装**
  版本 ≥ 8.0.RC2
- **CLI工具就绪**
  OpenCode、Claude Code等
- **网络连接**
  能访问GitCode仓库

云开发环境（推荐）

CANN社区云开发环境支持一键启动，NPU设备、CANN环境、Opencode工具等环境依赖均已默认安装



安装方式一：脚本安装（推荐）

A 方式A：项目级安装 (推荐)

在项目目录下安装，配置仅对当前项目生效，**适合多项目开发场景**

```
# 1. 克隆CANN Skills仓库
git clone https://gitcode.com/cann/cannbot-skills.git
# 2. 进入算子直调
cd cannbot-skills/plugins-official/ops-direct-invoke
# 3. 执行初始化脚本（OpenCode用户）
bash init.sh project opencode
# 3. 执行初始化脚本（Claude用户）
bash init.sh project claude
```

B 方式B：全局安装

在用户目录下安装，配置全局生效，**适合单一项目开发场景**

```
# 1-2步同上（克隆仓库并进入目录）
# 3. 执行初始化脚本（全局）
bash init.sh global opencode
# Claude用户
bash init.sh global claude
```

init.sh 安装内容

Skills技能模块

算子开发Skills

Agents配置

智能代理定义文件

asc-devkit工具包

算子开发工具集

配置文件

.opencode/config.json / CLAUDE.md

安装路径对比

项目级（OpenCode）	.opencode/
全局（OpenCode）	~/.config/opencode/
项目级（Claude）	.claude/
全局（Claude）	~/.claude/

安装方式二：手动安装

适用场景

按需安装：使用 npx skills add 从仓库选择 个 Skill 安装到当前项目，适用于 需 分能力的自定义 。

提示

init.sh（ 8 ）可一键安装 件全 Skills 与 Agents npx skills add 按需安装 个 Skill，二者 会在项目下生成配置目录（如 .opencode/），安装后可按需调整。

OpenCode用户安装步骤

```
# 1. 克隆仓库
git clone https://gitcode.com/cann/cannbot-skills.git
# 2. 进入克隆目录
cd cannbot-skills/
# 3. 按需安装（OpenCode）
npx skills add https://gitcode.com/cann/cannbot-skills.git
    --skill ascendc-env-check --agent opencode
```

Claude用户安装步骤

```
# 1. 克隆仓库
git clone https://gitcode.com/cann/cannbot-skills.git
# 2. 进入克隆目录
cd cannbot-skills/
# 3. 按需安装（Claude）
npx skills add https://gitcode.com/cann/cannbot-skills.git
    --skill ascendc-env-check --agent claude
```

环境校验与启动

目录结构校验

安装完成后，检查目录结构是否符合规范：

```
安装后在项目目录生成（OpenCode）
.opencode/
├── skills/ # 技能模块（知识能力）
├── agents/ # 角色代理（任务执行）
├── workflows/ # 工作流（阶段编排）
└── cannbot-manifest.json # 安装清
AGENTS.md # Agent 配置（位于项目根目录）
```

启动CLI工具

在初始化完成的目录下执行以下命令启动交互界面：

```
# 启动OpenCode CLI
opencode

# 启动后，输入算子开发需求
帮我开发一个abs算子，支持float16..
```

校验检查项

- **Skills目录存在**
  .opencode/skills/ 目录不为空
- **Agents目录存在**
  .opencode/agents/ 目录不为空
- **配置文件正确**
  AGENTS.md 和 cannbot-manifest.json 存在
- **CLI能正常启动**
  opencode 命令无报错

常见问题

如何查看帮助？

bash init.sh --help

如何更新Skills？

重新执行init.sh即可

安装失败怎么办？

检查CANN版本和网络连接

目录

Part 1 CANNBot概述

Part 2 环境搭建

Part 3 核心工作流

Part 4 实操演示

https://gitcode.com/cann/cannbot-skills

CANN

Ascend C：七阶段开发工作流



质量控制

每个阶段完成后才能进入下一阶段，确保开发质量

迭代优化

阶段4-5支持循环迭代，持续优化算子性能

Agent驱动

每个阶段由专门Agent执行，自动完成各项任务

工作流触发：所有算子开发任务会自动加载 算子开发 Agent（architect / developer / reviewer），按阶段执行。详见 AGENTS.md 配置文件。

CANN

阶段1-2：环境检查与需求分析

1 环境检查

确保开发环境就绪

NPU设备查询

检查NPU是否可用，获取设备信息

CANN环境验证

检查CANN版本、环境变量配置

依赖检查

验证必要的库和工具是否安装

```
# 自动执行环境检查
ascendc-env-check skill
```

2 需求分析

理解算子功能和要求

算子功能理解

分析算子的数学表达式和计算逻辑

数据类型确定

支持的数据类型（float16/float32等）

Shape分析

输入输出Tensor的Shape范围

性能要求

吞吐量、延迟等性能指标

Agent自动化：CANNBot的 **architect** Agent会自动分析需求，调用 `npu-arch` 和 `ascendc-api-best-practices` Skills，生成设计方案。

阶段3：算子实现 - Tiling设计

什么是Tiling？

Tiling是将大规模数据计算任务 **切分成小块**，分配到多个AI Core并行执行的技术。合理的Tiling策略能显著提升算子性能。

Tiling设计方法论

1 分析数据特征

理解输入数据的Shape、数据类型、访问模式

2 确定切分策略

选择合适的切分维度（N/H/W/C等）

3 计算Tiling参数

block_dim、tile_size等参数计算

4 Buffer规划

合理分配GM、UB、L1内存

自动Tiling设计：CANNBot根据算子类型自动选择合适的Tiling方案：Element-wise算子按数据块均匀切分，Reduce算子按Reduce轴切分，MatMul算子采用分块矩阵乘法策略。调用 ascendc-tiling-design Skill自动生成。

Tiling数据结构

```
// Tiling参数示例
struct AddTiling {
    uint32_t blockDim; // 核数
    uint32_t tileSize; // 块大小
    uint32_t totalSize; // 总大小
    uint32_t splitAxis; // 切分轴
};
```

切分策略

多核切分

将数据分配到多个AI Core

UB切分

适配Unified Buffer容量

阶段3：算子实现 - Kernel开发

Kernel代码生成

CANNBot自动生成符合 **Ascend C规范** 的Kernel代码，包括数据搬入、计算、数据搬出三个核心步骤。

代码生成流程

1. **分析需求**
   理解算子功能和数据特征
2. **选择模板**
   基于算子类型选择代码模板
3. **生成代码**
   填充模板生成完整Kernel代码
4. **优化调整**
   应用最佳实践和性能优化

代码质量保证

- 遵循API最佳实践
- 符合NPU架构特性
- 自动内存对齐
- 流水线优化

developer Agent自动完成

产出物结构

```
ops/{operator}/
├── CMakeLists.txt
├── {operator}_kernel.asc
├── build.sh
├── run.sh
├── scripts/
├── docs/
│   ├── DESIGN.md
│   └── PLAN.md
└── tests/
```

Kernel函数结构

```
// Kernel函数结构
extern "C" __global__ __aicore__ void
abs_kernel(...) {
    // 1. 数据从GM搬入UB
    DataCopy(xUB, xGM, size);
    // 2. 执行计算（Abs）
    Abs(zUB, xUB, size);
    // 3. 数据从UB搬出到GM
    DataCopy(zGM, zUB, size);
}
```

阶段4-5：验证优化与问题处理

4 验证与优化

确保算子正确性和性能

编译算子

使用build.sh编译生成算子二进制文件

运行测试

执行run.sh验证算子功能正确性

性能分析

使用profiling工具分析性能瓶颈

```
# 编译和测试
bash build.sh
bash run.sh
```

5 问题处理

调试和修复问题

精度调试

对比CPU和NPU输出，定位精度问题

Skill: ascendc-precision-debug

运行时调试

解析错误码，排查Kernel挂起等问题

Skill: ascendc-runtime-debug

自动修复

CANNBot自动诊断并提供修复建议

迭代优化：阶段4-5支持循环迭代，持续优化直到满足要求

智能诊断：Agent自动分析问题并提供解决方案

阶段6-7：性能验收与总结

6 性能验收

采集性能数据并判定达标状态

性能数据采集

调用msprof op采集8个CSV指标文件

性能达标判定

对照性能标准判定是否达标

瓶颈定位与优化

定位瓶颈类型并给出优化建议

最终判定

PASS　PASS WITH NOTES　FAIL

7 完成总结

汇总开发结果并生成文档

总分与代码路径

性能概要

Task Duration、主导流水、达标状态

关键问题列表

文档归档

DESIGN.md / PLAN.md / REVIEW.md / WALKTHROUGH.md

性能验收流程：调用 ops-profiling Skill → 数据归档到perf/目录

目录

Part 1 CANNBot概述

Part 2 环境搭建

Part 3 核心工作流

Part 4 实操演示

https://gitcode.com/cann/cannbot-skills

CANN

实操场景设定

算子需求

开发一个 **Abs（绝对值）算子** ，用于计算输入张量每个元素的绝对值。这是深度学习中的基础算子，广泛应用于各种神经网络模型。

数学表达式

$$y = |x|$$

计算输入张量的绝对值

输入输出示例

```
// 输入数据
x = [-1.5, 2.0, -3.5, 4.0]
```

```
// 输出结果
y = [1.5, 2.0, 3.5, 4.0]
```

预计耗时

1小时

技术要求

数据类型	float16
计算精度	1e-3

支持的Shape

[1, 128]　[4, 2048]　[32, 4096]

开发流程预览

1. 启动CANNBot并输入需求
2. 自动设计方案与代码生成
3. 代码审查与质量保障
4. 编译运行与验证

Step 1：启动CANNBot并输入需求

启动CLI工具

在初始化完成的目录下执行opencode命令，启动交互式开发环境：

```
# 进入项目目录
cd cannbot-skills/plugins-official/ops-direct-invoke
# 启动OpenCode CLI
opencode
# 成功启动后显示
OpenCode > _
```

输入算子开发需求

在交互界面中输入以下算子开发需求：

```
帮我开发一个abs算子，支持float16数据类型，shape主要是[1,128]、[4,2048]、[32,4096]
```

需求要素解析

算子类型	数据类型	Shape范围
Abs（绝对值）	float16	[1,128] [4,2048] [32,4096]

CANNBot响应流程

1. **自动加载工作流**
   加载算子开发 Agent 合
2. **确认需求理解**
   Agent复述需求确保理解正确
3. **进入阶段0**
   开始环境检查

提示技巧

- ✓ 描述要清晰具体
- ✓ 明确数据类型和Shape
- ✓ 可以补充性能要求

Agent驱动，自动执行

Step 2：自动设计方案与代码生成

自动设计方案

CANNBot的 **architect** Agent自动完成Tiling设计和方案规划：

Tiling策略

按数据块均匀切分，适配多核并行

Buffer规划

合理分配GM、UB内存空间

并行策略

利用多个AI Core并行计算

API选择

使用AscendC内置Abs高阶API

代码生成过程

1. 分析Abs算子特性（Element-wise）
2. 选择Element-wise算子代码模板
3. 填充Tiling参数和Shape信息
4. 生成Kernel、Tiling、Host代码

代码特点

- 符合AscendC编程规范
- 使用高阶API提升性能
- 自动内存对齐和优化
- 完整的编译和运行脚本

产出物结构

```
operators/abs/
├── CMakeLists.txt
├── abs_kernel.asc
├── build.sh
├── run.sh
├── scripts/
├── docs/
│   ├── DESIGN.md
│   └── PLAN.md
└── tests/
```

自动生成的代码示例

```
// Abs Kernel核心代码
Abs(zUB, xUB, size);
```

CANNBot自动生成符合Ascend C规范的代码，包含完整的数据搬入、计算、数据搬出流程。

Step 3：代码审查与质量保障

自动代码检视

CANNBot的 **reviewer** Agent自动进行多维度代码审查，确保代码质量和性能。

5大类别规范

代码风格
命名规范、缩进、注释

性能
内存访问、并行度、流水线

安全
内存越界、空指针检查

可维护性
模块化、可读性、可测试性

正确性
算法实现、边界条件

检视维度

- 代码风格规范
- 性能优化点
- 安全性检查
- 可维护性
- 正确性验证

检视内容示例

内存对齐检查 通过
检查UB内存是否32字节对齐

API使用规范 通过
验证API调用是否符合最佳实践

流水线优化 通过
检查是否使用双缓冲和流水线

reviewer Agent自动完成

Step 4：编译运行与验证

编译算子

使用生成的build.sh脚本编译算子：

```
# 进入算子目录
cd operators/abs
# 执行编译脚本
bash build.sh
# 编译成功输出
[100%] Built target abs_custom
```

运行测试

使用run.sh脚本运行测试：

```
# 执行测试脚本
bash run.sh
# 测试成功输出
[INFO] Input: [-1.5, 2.0, -3.5, 4.0]
[INFO] Output: [1.5, 2.0, 3.5, 4.0]
[INFO] Expect: [1.5, 2.0, 3.5, 4.0]
[PASS] Test passed!
```

产出物说明

docs/DESIGN.md、PLAN.md
需求设计文档、开发计划

abs_kernel.asc
Kernel实现代码

test/
测试用例代码

build.sh / run.sh
编译和运行脚本

验证要点

- 功能正确性
- 精度达标（< 1e-3）
- 性能满足要求
- 多Shape支持

恭喜！你的第一个算子开发完成！

终端

opencode

帮我开发一个 abs 算子，支持 float16 数据类型，shape 主要是 [1,128]、[4,2048]、[32,4096]

Build GLM-5.1 Zhipu AI Coding Plan

tab agents ctrl+p commands

• Tip Press Ctrl+X G or /timeline to jump to specific messages

现在，开始您的CANNBot之旅！
让Ascend C算子开发触手可及！

CANNBot通过Agent驱动的方式，将复杂的算子开发流程化繁为简。
从环境搭建到算子运行，全程自动化：
设计方案、编写代码、审查质量。

https://gitcode.com/cann/cannbot-skills

CANNBot Skills**开源交流群点**击如下链接获取

https://gitcode.com/cann/cannbot-skills/discussions/2



快速上手

1小时开发第一个算子



开源社区

共建昇腾AI生态



持续成长

从入门到精通

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态
社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货　关注CANN公众号获取资讯

https://gitcode.com/cann/cannbot-skills